

SVT COMMUNICATION · AOÛT 2026

DU SCAN TERRAIN AU RAPPORT CLIENT

Stock UC, l'usine à rapports, le contrôleur universel de girouettes, l'exploitation en direct — une chaîne d'outils construite de bout en bout pour SVT.

TERRAIN

DONNÉES

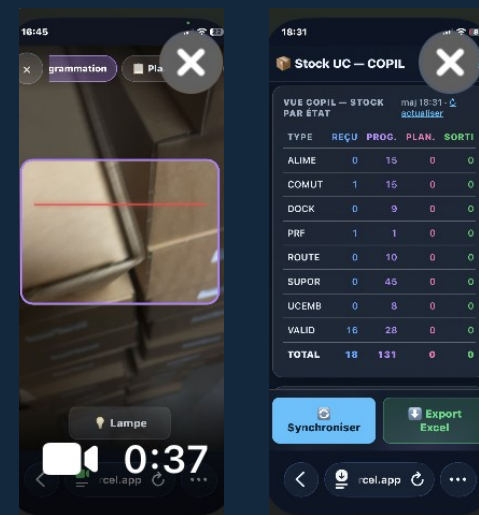
MATÉRIEL

RAPPORTS

Julien Abdou Gonzales · portfolio & proposition de stage



Girouette Hanover réelle, pilotée par l'ESP32 SVT — sans logiciel vendeur.



Stock UC en production —
scan caméra et vue COPIL,
sur les téléphones de l'équipe.

De l'idée de Josué à la poche de l'équipe

Scanner les entrants et sortants pour prévenir les manques en fin de chantier — et suivre le passage « reçu → programmé » demandé pour le COPIL.

Le besoin — compter les équipements billettiques à l'arrivée (UC, valideurs, switches, routeurs, alimentations...), puis les suivre d'état en état jusqu'à la pose.

La réponse — une web app zéro installation, ouverte depuis n'importe quel téléphone — caméra, douchette USB/Bluetooth ou saisie manuelle, en rafale.

Partagée — tous les téléphones parlent à la même base : la vue COPIL et le journal sont identiques pour toute l'équipe, en direct. PIN d'équipe vérifié côté serveur.

Traçée — chaque changement d'état est daté et journalisé ; un mode Suppression retire un code en gardant la trace du mouvement.

LE CYCLE DE VIE, EN COULEURS D'ÉTAT

Reçu



Programmé



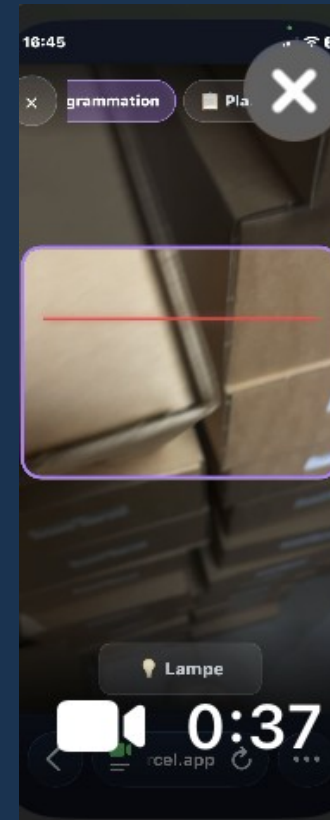
Planifié



Sorti

Défectueux

accessible depuis n'importe quel état · re-scan = flash plein écran de la couleur de l'état courant



L'UI de scan, pensée techniciens

plein écran, fenêtre de visée, ligne laser, lampe torche, écran maintenu allumé — et les chips d'étape en haut pour changer de mode sans quitter le scan.

Sous le capot : simple dehors, sérieux dedans

Une app de terrain n'a pas le droit de perdre un scan, ni d'accepter un code fantôme.

● MOTEUR DE SCAN

Décodeur — détecteur natif du navigateur (Android), sinon ZXing C++ compilé en WebAssembly, embarqué dans le fichier.

Performance — décodage pleine résolution 1920×1080, ~9 img/s — ~0,4 s pour lire un code sur iPhone.

● BASE PARTAGÉE

Serveur — API serverless Vercel + Upstash
Redis — 6 endpoints (scan, stock, journal, item, health, cleanup).

Accès — PIN d'équipe (header x-svt-pin) vérifié côté serveur, mémorisé sur l'appareil.

● ROBUSTESSE

Hors-ligne — file locale + envoi automatique au retour du signal — on ne perd jamais un scan.

Anti-fantômes — 8 préfixes canoniques (UCEMB, VALID, ROUTE...) ; codes normalisés, sans préfixe = refus net.

● COPIL & EXPORT

Vue COPIL — tableau type x état avec totaux, rafraîchi après chaque scan et toutes les 30 s.

Excel — 3 feuilles — Stock (dates par état), COPIL (matrice), Journal (mouvements).

● QUALITÉ PROUVÉE

Tests — 14 tests API + 17 tests e2e Playwright, tous verts (harnais Express + mock Redis).

Bugs traqués — 2 bugs caméra diagnostiqués et corrigés, preuve à l'appui (zone de scan largeur 0, décodage basse résolution).

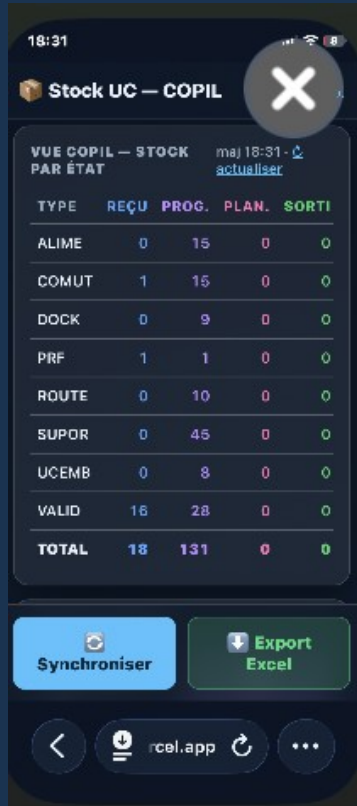
● ERGONOMIE TERRAIN

Retour haptique — double coup = déplacé, triple = refusé ; flash plein écran couleur d'état au re-scan.

Design — SVT Glass — la base UI commune à tous les outils SVT (design/svt-glass.css).

En production — adopté pendant les réceptions

Capture du terrain, 6 août : la vue COPIL en direct sur un téléphone de l'équipe.



18:31

Stock UC — COPIL

VUE COPIL — STOCK
PAR ETAT

maj 18:31 - actualiser

TYPE	REÇU	PROG.	PLAN.	SORTI
ALIME	0	15	0	0
COMUT	1	15	0	0
DOCK	0	9	0	0
PRF	1	1	0	0
ROUTE	0	10	0	0
SUPOR	0	45	0	0
UCEMB	0	8	0	0
VALID	16	28	0	0
TOTAL	18	131	0	0

Synchroniser Export Excel

< rcel.app >

149

équipements suivis
en temps réel

8

types d'équipements,
chacun sa ligne COPIL

131

déjà passés
en « Programmé »

0

scan perdu —
file hors-ligne

Demandée par le terrain — l'app répond à un besoin exprimé par Josué — le suivi du matériel entrant/sortant — et au passage « reçu → programmé » attendu au COPIL.

Déployée en vrai — en ligne sur svt-communication.vercel.app, partagée à l'équipe par simple lien — aucun déploiement de flotte, aucune installation.

Utilisée en rafale — réception scannée carton par carton, la vue COPIL se remplit en direct ; l'export Excel sort la synthèse en un geste.

Prochaines pistes — import d'une liste attendue (CSV) pour pointer les manquants par chantier ; historique détaillé par équipement au tap.

Une chaîne d'outils qui se répond

Chaque brique nourrit la suivante : le terrain produit des preuves, les preuves font les rapports, les rapports éclairent le COPIL, le COPIL pilote le terrain.

TERRAIN

Stock UC

scan & suivi du matériel

Workbench SMS RUT241

agent piloté par texto en LTE

Oracle caméra Mobitec

expérimentation autonome

MAJ des UC

3 unités en parallèle

DONNÉES

Couche Synchroteam

67 932 interventions miroir

Dashboard COPIL

carte site-first, 899 sites

Recherche métier

zéro identifiant à mémoriser

Serveur MCP

lecture seule pour les IA

RAPPORTS

Usine à rapports

preuves → études DOCX

Cockpit /rapport

le document est l'interface

Moteur 3D headless

planches d'implantation

Vocabulaire miné

363 Mo de rapports validés

MATÉRIEL

Contrôleur girouettes

Hanover · Mobitec · Lumiplan

Firmware ESP32

double bus RS485

Carte KiCad v1

vers un boîtier de série

FPGA KV260

compétence vidéo en cours

le volant : terrain → preuves → rapports → COPIL → terrain

L'usine à rapports : preuves → études DOCX

Le livrable le plus coûteux de l'entreprise — les Études véhicules pour des clients comme KEOLIS — généré depuis les preuves, relu par des humains.

34 600

lignes de code
(Flask + React)

64

routes API
sur 18 fichiers

363 Mo

de rapports validés,
minés pour leur langue

18

suites de tests
pytest

100%

des valeurs IA traçables
jusqu'à une preuve

SYNCHROTEAM → PAQUETS DE PREUVES → MAPPING IA (OPENROUTER) → PORTES DE CONFORMITÉ → DOCX

Le document est l'interface — le cockpit /rapport se lit comme l'étude elle-même : chaque champ est un contrôle en ligne, sa preuve s'affiche à côté, la conformité s'annonce avant l'export.

IA ancrée, jamais crue — chaque valeur générée porte ses identifiants de preuve ; les suggestions sans ancrage sont mises en quarantaine — pas de preuve, pas de valeur.

Moteur 3D headless — les planches d'implantation sont rendues depuis des modèles 3D de bus (10 m et 18 m articulé) dans un navigateur sans écran, puis injectées dans le Word.

Contrats stables — les images passent par une API versionnée (implantation_docx_images_v1) — chaque système évolue librement sans casser l'autre.

La couche de données et le dashboard COPIL

Synchroteam sait « où et qui » — cette couche le rend cherchable, visuel, disponible en permanence. Et Synchroteam reste strictement en lecture seule.

67 932

interventions synchronisées
en continu

3 600

projets miroirés,
rafraîchis toutes les 60 s

899

sites sur la carte
« site-first »

24/7

runtime + Caddy + ngrok
sur la machine pag-win

0

identifiant COxxxx
à mémoriser

L'intuition site-first — le flux géolocalise le site, pas le véhicule — donc une pastille par site avec anneau d'état, qui éclate en interventions au clic. Navigation : Site → CO → bus.

Recherche métier classée — par site, client, technicien, véhicule, type ou famille (« billettique ») — un vrai moteur à scores, pas un grep.

Sites bloqués d'abord — anneaux rouges, chip « N sites à débloquer », deck de cartes trié blocages en tête.

La couche métier — les tables business_* portent ce que Synchroteam n'a pas : suivi COPIL, inventaire, véhicules, préparation des rapports, écarts — importées et lues depuis Google Sheets.

Règle zéro — rien n'écrit jamais dans Synchroteam ; le serveur MCP pour les assistants IA ouvre même la base en mode lecture seule au niveau de l'OS.

Une seule porte d'entrée — la passerelle ngrok + Caddy route 8 chemins publics vers 5 services locaux — un seul tunnel pour tout le parc d'outils.

Le contrôleur universel de girouettes

Trois protocoles, un boîtier SVT — capturer → décoder → rejouer → encapsuler : un playbook écrit, reproductible par tout dev SVT.



La preuve d'établi : bitmap arbitraire sur girouette Hanover réelle, émis par l'ESP32 — vidéo complète sur abyss8.github.io.

L'échelle — 365 fichiers · ~28 600 lignes Python + ~17 700 lignes C/C++ · espace KiCad v1 ouvert (alim véhicule 24 V, TVS, double RS485).

HANOVER

OPÉRATIONNEL

Bitmaps arbitraires sur panneau réel ; table de commandes rétro-conçue (init F000, ident 90MB, bitmaps 144×19 et 1360) — confirmée par deux implémentations indépendantes (la nôtre + Hanover_Diag, la suite d'un collègue intégrée avec sa provenance).

MOBITEC

TEXTE NATIF DÉCODÉ

Grammaire de trame percée par capture-diff (adresse, registres D0-D4, checksum somme & 0xFF) ; texte arbitraire synthétisé et vérifié sur panneau — y compris un test de falsification : une trame au checksum volontairement faux, correctement rejetée.

LUMIPLAN

CAPTURE & DÉCODAGE

Sessions à l'analyseur logique annotées action par action ; l'honnêteté est dans le code : STATUS = « investigation » — pas de pilote tant qu'un format de trame réel n'est pas vérifié sur matériel.

L'automatisation là où les camions se garent

Dépôts et bords de route ont rarement un Wi-Fi fiable — le LTE et une webcam suffisent.

WORKBENCH SMS · RUT241

Le principe — un routeur LTE Teltonika RUT241 — le même matériel que SVT pose dans les bus — relaie les SMS vers un agent IA de développement sur la machine d'établi : un texto lance un projet, corrige des tests, demande un diff.

Sécurité par conception — liste blanche de numéros sur chaque chemin, jeton d'appairage, SMS sortants en simulation par défaut, journaux expurgés des secrets.

Sans dépendance — bibliothèque standard Python uniquement — rien à auditer ; exposé aux agents locaux comme serveur MCP (5 outils).

Prouvé — 43 tests unitaires, tous verts — vérifiés cette semaine.

ORACLE CAMÉRA · MOBITEC

Le problème inversé — le défilement Mobitec n'est produisible par aucun outil vendeur sur notre panneau — impossible de sniffer ce qu'on ne peut pas provoquer. Alors la caméra devient l'oracle.

La boucle — la main émet une trame candidate → l'œil photographie le panneau → l'IA juge la photo (statique ? défilant ?) → l'essai suivant est choisi. 5 registres × 256 valeurs = 1 280 expériences, une nuit d'établi.

Discipliné — une seule variable change par essai ; reprise sur coupure de courant ; émission désarmée par défaut ; câblage vérifié avant toute trame.

Comprendre chaque boîtier du bus

Les girouettes ne sont qu'une branche du système embarqué — la même discipline s'étend à la chaîne billettique entière.

MAJ SEMI-AUTOMATISÉE DES UC

Disque USB virtuel — un ESP32-S3 présente les mises à jour à l'UC comme une clé USB en lecture seule — bascule OFF / M0 / M1 par série, bouton ou HTTP, état persistant.

Trois UC en parallèle — chaque ESP32-P4 isole le réseau de son UC : les trois gardent la même IP par défaut, et l'opérateur les pilote ensemble.

Cockpit navigateur — un terminal xterm.js par UC, commande diffusée à tous (« Send All »), reconnaissance des écrans de menu pour un passage étape par étape.

RECONNAISSANCE UC

Lecture seule, par playbook — apprendre l'UC d'établi en SSH sans en modifier un octet ; autotests et états de flotte (En ligne / Hors ligne / Dégradé) déjà observés — la base d'un futur outillage de diagnostic SVT.

COMPÉTENCE EN CONSTRUCTION · FPGA

Kria KV260 — pipeline vidéo caméra → Sobel dans la logique programmable → HDMI ; le modèle de référence exige une correspondance pixel à pixel — la falsification est non négociable.

Les petites briques qui font tenir le reste

Chaque friction du quotidien a reçu son outil — petit, documenté, en service.

● INVENTAIRE ILLUSTRÉ

333 références SVT jointes aux références RS Components, 111 photos produits hébergées en URLs stables (GitHub Pages) — les formules IMAGE() de Google Sheets ne cassent plus jamais.

● SUIVI DE PROGRESSION

les feuilles de temps Eurecia générées depuis les preuves de travail réelles : 113 éléments de preuve → 19 lignes proposées, chacune avec citations, score de confiance et commentaire Problème → Action → Apprentissage.

● PASSERELLE UNIQUE

une Caddyfile de 43 lignes remplace un proxy maison : 8 chemins publics → 5 services locaux, un seul tunnel ngrok pour démontrer tout le parc d'outils.

● SVT GLASS

le design system commun — fond nuit de dépôt, panneaux givrés, un accent cyan, et la palette d'états du stock partagée par tous les outils. Un seul fichier CSS à copier.

Une signature d'ingénierie, la même partout

Ce qui relie un scanner de codes-barres, un protocole RS485 et un pipeline de rapports.

● LA PREUVE AVANT LA CONFIANCE

modèle de référence pixel à pixel sur FPGA ; trame au checksum volontairement faux pour vérifier que le panneau refuse ; caméra comme oracle quand le protocole ne peut pas être sniffé ; 14 + 17 + 43 tests automatisés sur les outils de terrain.

● LA SÉCURITÉ PAR DÉFAUT

émissions RS485 et SMS en simulation tant qu'on n'arme pas explicitement ; listes blanches ; secrets expurgés des journaux ; et la règle zéro — les systèmes de production (Synchroteam) restent en lecture seule.

● DES PLAYBOOKS, PAS DES EXPLOITS

protocole girouettes, reconnaissance UC, mise en service Stock UC : chaque méthode est écrite pour être répétée par un autre dev SVT. Le savoir est un actif de l'entreprise, pas d'une personne.

● L'IA COMME LEVIER, L'INGÉNIEUR RESPONSABLE

Claude Code et les agents accélèrent chaque brique — mais chaque valeur générée est traçable, chaque sortie relue, chaque accès en lecture seule. L'outillage IA est un multiplicateur, pas un substitut.

Des chiffres honnêtes, déjà mesurables

12

dépôts de code livrés
pour l'écosystème SVT

95 000+

lignes de code sur la chaîne
complète (web, Python, C/C++)

3 → 1

outils d'édition vendeurs
vers un seul flux SVT

149

équipements suivis en direct
dès la première réception

Visibilité — le COPIL voit le stock, les sites et les blocages en direct — sans dépendre d'un tableur tenu à la main.

Marge — le débit d'études monte sans embauche : la rédaction devient une relecture guidée, les erreurs sont attrapées à la porte.

Indépendance — la capacité de SVT à servir ses contrats ne dépend plus du bon vouloir logiciel de trois vendeurs de girouettes.

Position — d'installateur à intégrateur-constructeur : l'entreprise qui comprend le véhicule plus profondément que les revendeurs.

Transmissible — playbooks écrits, design system commun, contrats d'API versionnés — un nouveau dev SVT est productif vite.

Mesuré — chaque affirmation embarque sa métrique : lots pilotes, chronométrages avant/après, statistiques de porte.

La suite : un stage d'ingénieur systèmes

Tout ce qui précède a été construit en marge du poste actuel. Un cadre de stage transforme cet élan en résultats planifiés, mesurés et transmis.

QUATRE MISSIONS CONCRÈTES

- 1 Stock UC v2**
import des listes attendues (CSV) pour pointer les manquants par chantier ; historique par équipement ; bilan chiffré après 3 réceptions.
- 2 Girouettes : le pilote de référence**
fusionner notre table Hanover avec Hanover_Diag, sortir Mobitec du mode gabarit — premier pilote du contrôleur sur site réel.
- 3 L'usine à rapports en production**
passer /rapport sur les contrats d'études en cours et mesurer les heures-par-étude, avant / après, sur un lot pilote.
- 4 Flux & santé rapport au COPIL**
brancher la vue cycle-de-vie et la santé des rapports dans la routine COPIL — les deux questions que le comité pose vraiment.

POURQUOI C'EST UNE BONNE AFFAIRE

Continuité — les outils tournent déjà — le stage capitalise, il ne repart pas de zéro.

Cadre — objectifs écrits, points d'étape réguliers, livrables mesurés (le suivi de progression existe déjà, preuves à l'appui).

Transfert — chaque mission se conclut par un playbook ou une formation courte à l'équipe — le savoir reste chez SVT.

Risque nul — toutes les disciplines déjà démontrées : lecture seule sur la production, simulation par défaut, tests verts.

LES OUTILS TOURNENT DÉJÀ.

Un cadre officiel les fait passer à l'échelle — au-delà d'une seule personne.

Merci — place aux questions et à la démonstration en direct.

Julien Abdou Gonzales · julienabdougonzales@gmail.com · abyss8.github.io · github.com/abyss8

Stock UC : svt-communication.vercel.app — scannez, la vue COPIL suit.